Right-Wall Following with Tabular Q-Learning and SARSA

Shivraj Singh Bhatti

***Abstract*— We implemented a ROS/Gazebo right-wall follower with two tabular RL agents: Q-learning and SARSA. The final controller uses a 72-state local-geometry encoder built from front clearance, right-wall distance, wall heading, and front-left/front-right opening bits. This replaces an 18-state encoder that aliased corridor motion, U-turn mouths, and blocked-front recovery cases. The action set has five commands: two soft turns, two hard turns, and straight motion. Both algorithms use $\alpha = 0.15$, $\gamma = 0.95$, and ϵ decay from 0.20 to 0.03, with training capped at 360 episodes of 240 steps at 6 Hz. Quantitative reward curves and checkpoint metadata show that both methods learn effective policies, but they also show that reward alone is not a complete selection metric. Early reward settings encouraged failure behaviors, including drifting through openings, local pivot loops, and a SARSA right-turn sweep in U-turn recovery. We fixed these issues by retuning `too_far` reward ranking, tightening evaluation and checkpoint selection, fixing test-spawn timing in Gazebo, and adding a SARSA test-only deadlock breaker. The final package supports reproducible training and greedy evaluation for both algorithms, with saved checkpoints in `config/` and demos covering all five required wall-following situations.**

I. Introduction

Wall following is a useful benchmark for tabular reinforcement learning because the robot must react to local geometry, not just follow a single straight corridor. In this project, Triton must stay close to the right wall and still handle openings, bends, dead ends, and U-turn-like geometry. We implemented two classic RL methods, Q-learning and SARSA, inside a ROS Noetic package running in Gazebo.

The key requirement was recovery behavior across the five required scenarios, not only high episode reward. During development, some checkpoints scored well while still drifting through open space, turning in place, or missing U-turn recovery. Therefore, we focused on state design, action semantics, reward ranking, and fixed test-time evaluation.

The final package exposes both algorithms in training and testing modes. Training uses near-wall seed poses and learns a tabular policy from scratch or from an existing checkpoint. Testing loads the best learned policy and runs greedily from arbitrary spawn points. We also fixed two infrastructure issues that initially contaminated the results: a manual test-start race in Gazebo and a SARSA-only local deadlock in test mode.

II. Approach

A. Sensing and State Encoding

The final D2 encoder uses a compact local-geometry representation with 72 states:

$$2 \times 3 \times 3 \times 2 \times 2 = 72.$$

The five factors are:

- `front_bin`: blocked or clear
- `right_bin`: too close, good, or too far
- `heading_bin`: toward wall, parallel, or away from wall
- `front_left_open_bin`: open or closed
- `front_right_open_bin`: open or closed

This encoder divides the laser field of view into local sectors that are meaningful for right-wall following. In the old 18-state design, right alignment (heading) was estimated from the difference between right-front and right-rear minimum ranges, i.e., `rf\_min - rr\_min`; this proxy is preserved in `heading\_bin` but is now disambiguated by opening bits. The current thresholds in `scripts/state_encoder.py` are:

- front sector: 75° to 105°
- right-front sector: 15° to 65°
- right-rear sector: -70° to -20°
- front-left open sector: 100° to 155°
- front-right open sector: 25° to 85°
- front blocked threshold: $0.55\,\mathrm{m}$
- right too-close threshold: $0.55\,\mathrm{m}$
- right too-far threshold: $0.95\,\mathrm{m}$
- opening threshold: $1.20\,\mathrm{m}$

That state design was chosen because it separates corridor tracking from the two failure cases that caused the most trouble: dead-end/U-turn geometry and wall loss through open space.

B. Action Space

The D2 controller uses five discrete actions:

- `turn_left_hard`
- `turn_left_soft`
- `straight`
- `turn_right_soft`
- `turn_right_hard`

The actions are intentionally asymmetric in their kinematics. `straight` has the largest forward motion. Soft turns combine forward motion with moderate angular correction. Hard turns are near-pivot actions with zero forward speed. That choice helped with corner commitment, but it also created the possibility of a zero-progress loop if the reward function made hard turns too attractive.

Figure 2 summarizes the encoder geometry, the 18-state aliasing failure cases, and the five-action kinematic design used in both Q-learning and SARSA.

C. Q-Table Definition and Initialization

The Q-table is a dense mapping from state-action pairs to scalar values. If the input checkpoint is missing, the agent

starts from zeros. If the input file is an older 18-state table, it is expanded into the 72-state space by copying the old values across the two opening bits. After loading, any missing state-action entries are filled with zero so the table is dense and safe for updates.

That initialization strategy matters for two reasons. First, it makes training reproducible from a clean start. Second, it lets a legacy checkpoint bootstrap the final D2 state space instead of throwing away all prior learning.

D. Learning Rules

For Q-learning, the update is the standard off-policy rule [1]:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big].$$

For SARSA, the update is on-policy and follows the standard on-policy TD control form from introductory RL treatments [2], [3]:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma Q(s',a') - Q(s,a)\Big].$$

Both methods share the same state/action/reward definitions and hyperparameters; the only algorithmic difference is the target term in the update.

The final hyperparameters were:

- $\alpha = 0.15$
- $\gamma = 0.95$
- ϵ decays from 0.20 to 0.03
- control rate: 6 Hz
- max episodes: 360
- max steps per episode: 240

Training runs use near-wall seed poses and disable the wall-acquisition pre-controller so the TD updates reflect the learned policy rather than a hand-coded rescue behavior. Testing runs load the chosen best table and run greedily. A manual test-start option teleports the robot to named edge-case poses, but the pose application now waits until the Gazebo model exists so the test location is deterministic.

E. Reward Shaping

The reward function combines front clearance, right-wall distance, heading quality, forward progress, and state-action-specific corrections. The final reward design evolved through several rounds of failure analysis. In particular:

- `clear+good+parallel` should strongly favor `straight`
- `clear+good+toward_wall` should favor `turn_left_soft`
- `clear+good+away_from_wall` should favor `turn_right_soft`
- `right_bin = too_far` should prefer a forward-carrying right correction, not a pivot in place
- blocked-front states should still reward the appropriate escape turn

The final tuned rewards in `config/d2_params.yaml` were selected from repeated failure analysis. We specifically removed three failure behaviors: drifting away from the wall, overusing hard turns in good states, and a SARSA right-turn sweep that failed U-turn recovery.

III. Experimental Results

A. Quantitative Results

Figure 1 plots accumulated reward per episode with 10-episode rolling means. Table I summarizes the final training metadata recorded in the saved checkpoints.

Alg.	Eps.	Time	α/γ	ϵ	Ckpt.
Q-learning	376	6.1 h	0.15/0.95	0.20→0.03	`latest`
SARSA	560	17.9 h	0.15/0.95	0.20→0.03	`latest`

TABLE I. Final training summary from checkpoint metadata. Time is approximate wall-clock duration computed from saved run timestamps.

Fig. 1. Accumulated reward over training (raw curves faint; 10-episode rolling means bold). Generated with `scripts/plot_d2_metrics.py`.

The most important quantitative lesson is that training reward alone was not sufficient to select the final policy. We found multiple cases where a checkpoint had a better-looking curve but worse real behavior in the required scenarios. The final `best` policy was selected by fixed greedy evaluation and manual scenario testing, not by a single reward spike.

B. Qualitative Results

The assignment requires qualitative evidence that the learned policies handle the five wall-following situations. Figure 3 shows final trajectory afterimages for the required cases and makes turning direction and recovery behavior visible.

Qualitative evaluation helped identify failure modes that reward curves alone did not show. Q-learning produced a reliable greedy policy in the required scenarios. SARSA also learned a strong policy, but it showed a local pivot loop in `clear+too\_close` states. The final SARSA test configuration includes an opt-in deadlock breaker for recording; the learned table is unchanged.

IV. Comparison

Q-learning and SARSA converged to different behaviors under the same state/action/reward design. Q-learning was easier to debug because the max-over-actions target made greedy behavior clearer. SARSA was more sensitive to trajectory quality and exposed reward-model mistakes earlier.

Modeling choice	Observed behavior	Final fix
Too-far reward still paid enough for `straight` when the right wall was lost.	Skipped the U-turn mouth and drifted through open space instead of reacquiring the wall.	Penalize `straight` in too-far states; rank `turn_right_soft` above `turn_right_hard`.
Treating all right turns alike in U-turn geometry (`too_far`).	SARSA learned a large right-hand sweep instead of crossing the mouth cleanly.	Separate soft vs. hard right turns; reward the forward-carrying correction.
Hard turns with zero linear speed (near-pivot).	Local pivot loop under greedy test with no forward progress (SARSA especially).	Keep training semantics; add SARSA-only, test-only deadlock breaker after repeated zero-progress steps.
Manual spawn / test-start before the Gazebo model was ready.	Policy judged from the wrong physical start; looked worse than it was.	Wait for model presence before applying the named test pose.

TABLE II. Problem-structuring lessons from D2 iterations: each row links one design or evaluation choice to an observed failure and the fix used in the final system.

The main comparison points are:

- Q-learning was more forgiving during training, but its checkpoint selection still needed manual validation.
- SARSA was better at surfacing reward-model mistakes, especially the `too_far` U-turn recovery issue.
- Both methods needed the same state encoder and action set, but they responded differently to reward shaping.
- The final SARSA deadlock was not solved by retraining from scratch; it was solved by making the test harness treat repeated zero-progress states as a local loop.

Two implementation lessons were most important. First, moving from 18 to 72 states reduced aliasing at dead ends and openings. Second, evaluation infrastructure (launch defaults, test-start timing, and fixed scenario checks) had a large effect on measured behavior.

V. Requirement Coverage

This report addresses all required items for Deliverable 3:

- **Design details for Q-learning and SARSA:** sensing/FOV partition, state, action, reward, and Q-table setup are described in Section II.
- **Model parameters:** both methods use $\epsilon : 0.20 \rightarrow 0.03$, $\alpha = 0.15$, and $\gamma = 0.95$ with 6 Hz control, 360-episode cap, and 240-step episode cap.
- **Quantitative evidence:** reward-vs-episode curves (Fig. 1) and checkpoint summary (Table I) show learning progress and training cost.
- **Qualitative evidence:** final demo afterimages for all five wall-following situations are provided in Fig. 3.
- **Q-learning vs. SARSA comparison:** training episodes/time and behavior differences are summarized in Section IV and Table I.

VI. Conclusion and Future Work

This project shows that a compact local-geometry state space, a five-action controller, and tuned rewards are sufficient for robust wall following in ROS/Gazebo. The main result is a policy that works in required scenarios and an evaluation process that detects reward-hacking failures early.

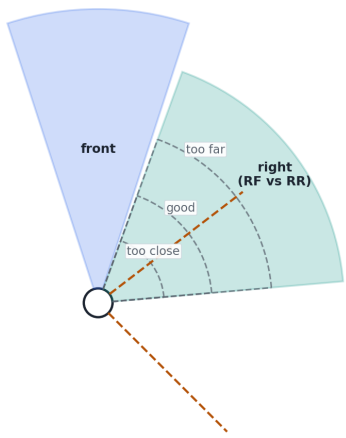
Future work could go in three directions. The first is better scenario coverage with an even more systematic evaluation suite. The second is richer motion primitives, such as a slightly forward-moving hard turn, if that is treated as a new action set rather than a mid-run tweak. The third is more principled automated checkpoint selection so that `best` is chosen from behavior-level evaluation rather than a single scalar reward signal.

References

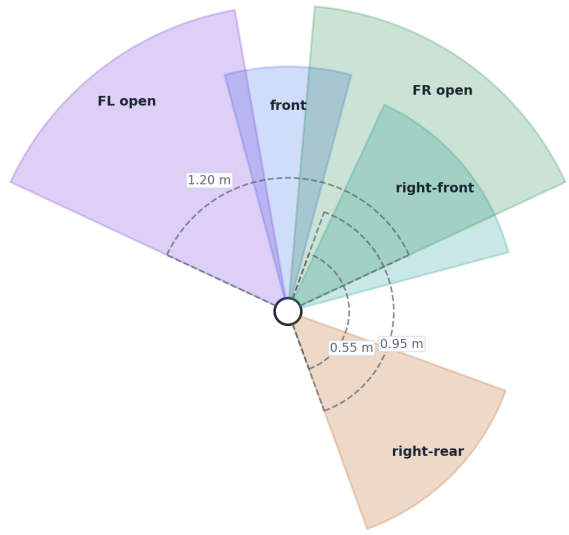
[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

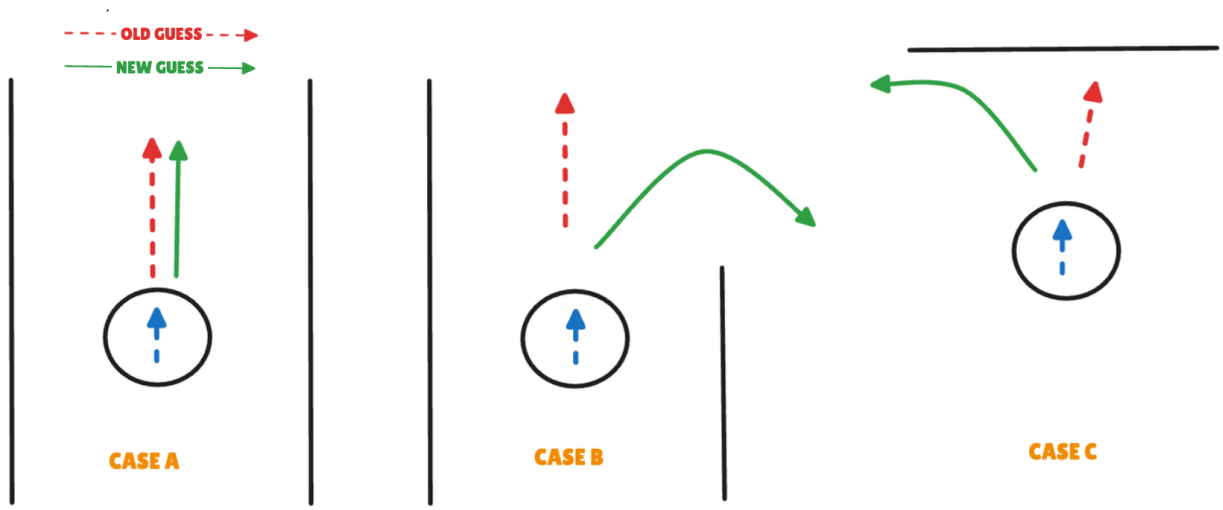
[3] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement Learning: A Survey," *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, 1996.



(a) Coarse 18-state geometry ($2 \times 3 \times 3$): `front\_bin`, `right\_bin`, and `heading\_bin`, where heading uses right alignment proxy `rf\_min - rr\_min`.



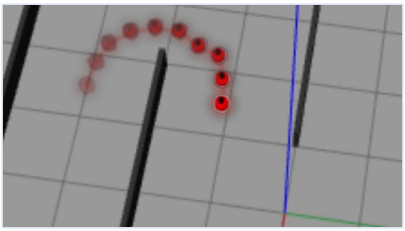
(b) Final 72-state encoder ($2 \times 3 \times 3 \times 2 \times 2$) with `front\_left\_open\_bin` and `front\_right\_open\_bin`.



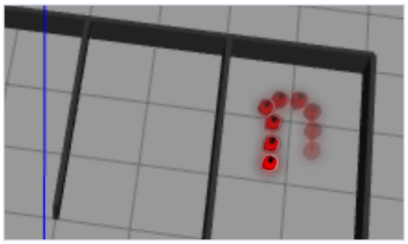
(c) Aliasing under `clear|good|parallel`: **Case A** normal corridor (straight is correct), **Case B** U-turn mouth/opening (should begin reacquisition around the lip), and **Case C** blocked-front recovery (should commit to a recovery turn).

(d) Five discrete actions (v_y, ω_z) used by D2: soft turns keep forward progress, while hard turns are near-pivot actions with v_y=0.

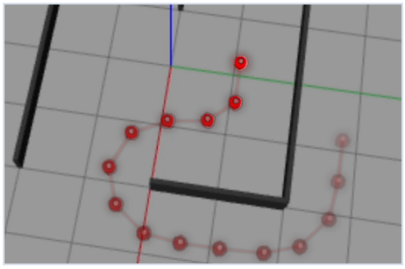
Fig. 2. State and action design for D2 wall following. **(a)** Coarse 18-state encoder. **(b)** Final 72-state encoder that adds opening bits and separates geometries that were previously aliased. **(c)** Three physically different layouts that collide under the coarse tuple `clear|good|parallel`; opening bits resolve this ambiguity. **(d)** Action kinematics used by both learning methods.



(a) I-beam / interior wall end.



(b) Dead-end / U-turn recovery.



(c) Right turn, 180° right U-turn, left turn.

Fig. 3. Qualitative evidence from final demos. **(a)** Interior wall-end (I-beam) handling. **(b)** Dead-end/U-turn-style recovery. **(c)** Composite sequence containing a right turn, a right U-turn, and a left turn. Together, these panels cover the five required wall-following situations.